LifeSync

AI-Powered Personal Wellness Assistant

Delta Design & Implementation Report

Author:	Fatma Zehra Paksoy
Date:	April 14, 2026
Version:	1.0
Review Date:	March 27, 2026
Reviewers:	Mustafa Gozutok, Mehmet Yasin Tosun, Mehmet Umur Oz
Repository:	github.com/beyzadegirmenci/LifeSync

0 Contents

1 Introduction **3**

2 Architecture at Review Time (Baseline) **3**
2.1 Layered Architecture . . . 3
2.2 Design Patterns at Baseline . . . 4
2.3 Known Weakness at Baseline . . . 4

3 Received Feedbacks from Peer Review **4**

4 Feedback Selection and Justification **5**
4.1 Available Implementation Budget . . . 5
4.2 Effort Estimation Method . . . 5
4.3 Estimation Table . . . 6
4.4 Rejection Justification . . . 6

5 Implementation of Selected Feedbacks **6**
5.1 F1 – AI Integration: Facade Pattern (`WellnessPlanFacade`) . . . 6
5.1.1 Feedback Context . . . 6
5.1.2 Design Decision . . . 7
5.1.3 Architecture Update . . . 7
5.1.4 Updated Component Diagram (Delta) . . . 9
5.1.5 Updated Class Diagram (Delta – new classes only) . . . 9
5.2 F2 – Broader Dashboard: Structured Plan Table and Excel Export . . . 10
5.2.1 Feedback Context . . . 10
5.2.2 Design Decision . . . 10
5.2.3 Updated Sequence Diagram – AI Plan Generation (Delta) . . . 11
5.3 F3 – Profile Data to AI Output Continuity . . . 11
5.3.1 Feedback Context . . . 11
5.3.2 Design Decision . . . 12
5.4 F3b – Strategy Pattern: Level-Specific AI Customization . . . 13
5.4.1 Feedback Context . . . 13
5.4.2 Design Decision . . . 13
5.4.3 Implementation . . . 14
5.4.4 Strategy Classes . . . 15
5.4.5 Quality Impact . . . 15
5.5 F4 – End-to-End AI Validation (`planValidator.js`) . . . 15
5.5.1 Feedback Context . . . 15
5.5.2 Design Decision . . . 16
5.6 F2b – Observer Pattern: Real-Time Notification System . . . 17
5.6.1 Feedback Context . . . 17
5.6.2 Design Decision . . . 17
5.6.3 Class Structure . . . 18
5.6.4 Database Schema Extension . . . 18
5.6.5 Implementation . . . 19
5.6.6 Frontend Notification System . . . 20
5.6.7 Supported Event Types . . . 20
5.6.8 Observer Pattern Sequence Diagram . . . 20

5.7 F7 – Documentation Alignment . . . 21
5.7.1 Feedback Context . . . 21
5.7.2 What Was Updated . . . 21

6 Architecture Summary: Before vs. After **22**

7 New Design Patterns Added **22**
7.1 Strategy Pattern (Enhanced at Delta Phase) . . . 22
7.1.1 Overview . . . 22
7.1.2 Class Structure . . . 22
7.1.3 Purpose . . . 23
7.1.4 Integration with Facade and Profile Continuity . . . 23
7.1.5 Fallback Recommendations . . . 24
7.2 Facade Pattern . . . 24
7.3 Observer Pattern . . . 25
7.4 Complete Design Pattern Summary (Post-Delta) . . . 25

8 Quality Factor Impact **25**

9 Effort Actuals vs. Estimates **27**

10 Bug Fix Log (Testing Phase) **27**

11 Conclusion **29**

A File Structure Reference **30**

1 Introduction

This report documents the design and implementation changes made to the **LifeSync** project following the peer review session conducted on March 27, 2026. The review was performed by an external team who evaluated the system's architecture, code quality, and implementation completeness.

LifeSync is a full-stack wellness assistant application that collects user health data, classifies users via a rule-based algorithm, and generates personalized diet and exercise plans using a locally hosted Large Language Model (LLM) via Ollama.

The technology stack at the time of the review was:

- **Frontend:** React 19 + Vite (port 5173)
- **Backend:** Node.js + Express 5 (port 5000)
- **Database:** PostgreSQL (port 5432)
- **AI Service:** Ollama LLM – `llama3.2:3b` (port 11434)

At the time of review, the system supported user registration/login, JWT-protected dashboard access, BMI calculation, a health survey, and a *prototype* AI recommendation module implemented as a separate Python client that was not integrated into the main web application.

This report covers:

1. All feedbacks received from the peer review
2. Selection rationale with effort estimations and cost calculations
3. Detailed implementation of each selected improvement
4. Architecture updates and their impact on design patterns
5. Quality factor analysis and test mapping
6. Effort actuals vs. estimates and a bug fix log

2 Architecture at Review Time (Baseline)

2.1 Layered Architecture

The system used a 3-tier Layered Architecture (Presentation / Business Logic / Data), with an optional External AI layer.

Table 1: Architecture layers at review baseline

Layer	Technology	Responsibility
Presentation	React 19 + Vite	User interface, form management, routing
Business Logic	Node.js + Express 5	Auth, classification, AI orchestration
Data	PostgreSQL	Persistent user data storage
External AI	Ollama (llama3.2:3b)	LLM-powered plan text generation

2.2 Design Patterns at Baseline

Table 2: Design patterns present at review time

Pattern	Class(es)	Purpose
Builder	`ProfileBuilder`	Step-by-step profile update payload construction with field validation
Builder	`SqlUpdateBuilder`	Dynamic SQL UPDATE query generation via method chaining
MVC	Controller + Model + Route	Separation of routing, business logic, and data access
Middleware	`JwtMiddleware`	Cross-cutting JWT authentication applied to protected routes

2.3 Known Weakness at Baseline

The primary structural weakness at review time was that the AI recommendation logic was implemented in a **separate Python client** (`client/` directory) and was **not integrated** into the main Express.js backend. The `DashboardController` had no AI generation capability; it only computed BMI. This meant the AI plan generation flow was not accessible through the web application's normal user flow.

3 Received Feedbacks from Peer Review

The review team identified the following improvement points across sections 6, 7, 9, and 12 of their report:

Table 3: Complete list of received feedbacks

#	Section	Feedback	Type
F1	§6, §7, §9	*"Tighter integration between the web system and the AI recommendation pipeline."* The AI logic existed only in a Python client, not in the web backend.	Architectural

(continued from previous page)

#	Section	Feedback	Type
F2	§6, §9	*"Broader dashboard capabilities."* The dashboard was limited to BMI display only; no structured health plan output.	Functional
F3	§9	*"Improving the continuity between stored profile data and AI-generated outputs."* User's stored profile (allergies, preferences, activity) was not fully reflected in the AI plans.	Functional
F4	§7	*"Richer end-to-end validation around AI-assisted flows."* If Ollama returned malformed output, the application would fail with no graceful recovery.	Quality
F5	§7	*"Stronger production-oriented deployment readiness."* No production configuration, no containerisation, no environment management.	DevOps
F6	§7	*"Expanded user-facing progress features."* No plan history, no tracking of wellness progress over time.	Functional
F7	§9	*"Refining documentation so that setup and implementation details remain aligned."* SDD diagrams did not reflect all implemented components.	Documentation

4 Feedback Selection and Justification

4.1 Available Implementation Budget

The remaining development window was approximately **4–5 working days** at **2 hours per day**, yielding a total budget of **8–10 person-hours**.

4.2 Effort Estimation Method

Three-point (PERT) estimation was applied:

$$E = \frac{O + 4M + P}{6} \tag{1}$$

where O = Optimistic, M = Most Likely, P = Pessimistic (all in hours). Cost was computed at a **student developer rate of €20/hour**.

4.3 Estimation Table

Table 4: PERT effort estimation and selection decision for all feedbacks

#	Feedback	O	M	P	E (hrs)	Cost	Selected
F1	AI integration (Facade)	3	4	6	4.2	€84	**Yes**
F2	Dashboard: plan table + export	2	3	4	3.0	€60	**Yes**
F3	Profile data ↔ AI continuity	1	2	3	2.0	€40	**Yes**
F4	End-to-end AI validation	1	2	4	2.2	€44	**Yes**
F5	Production deployment readiness	4	8	12	8.0	€160	**No**
F6	User progress tracking	5	10	16	10.2	€204	**No**
F7	Documentation alignment	1	1	2	1.2	€24	**Yes**
Total selected (F1+F2+F3+F4+F7)					**12.6 hrs**	**€252**	

4.4 Rejection Justification

F5 – Production Deployment Readiness: REJECTED

Estimated at **8.0 hours alone**. Setting up Docker containers, CI/CD pipelines, proper environment variable management, and reverse-proxy configuration would consume the entire remaining budget without adding visible functionality to the demo scenario. This is an infrastructure concern appropriate for a later project phase.

F6 – User Progress Tracking: REJECTED

Estimated at **10.2 hours**. Implementing plan history requires: (1) new database tables, (2) new API endpoints, (3) new frontend pages, and (4) state management changes. The scope far exceeds the remaining timeline and would risk destabilising the existing stable features.

5 Implementation of Selected Feedbacks

5.1 F1 – AI Integration: Facade Pattern (`WellnessPlanFacade`)

5.1.1 Feedback Context

The peer review noted in sections §6, §7, and §9:

> *"The most beneficial next step for LifeSync would be the gradual integration of the already existing AI recommendation logic into the main web application*

> *flow. Since the project already has both sides of the puzzle, a functioning web application and a meaningful recommendation prototype, the improvement path is quite natural."*

At review time, all AI-related logic was either in the Python client or scattered across approximately 120 lines inside `DashboardController`, with no separation of concerns.

5.1.2 Design Decision

The **Facade Pattern** was chosen to solve this problem. A new `WellnessPlanFacade` class was introduced to:

- Provide a *single, clean interface* for all AI plan operations
- Hide the complexity of prompt building, Ollama communication, validation, and retry logic from the controller
- Make AI logic independently testable via dependency injection
- Retire the Python client from the main flow entirely

Design Decision: Why Facade and not Strategy or plain Service?

A **Strategy** pattern would be appropriate if the AI model selection algorithm needed to change at runtime. A plain **Service** class would work but would not formally address the "hiding subsystem complexity" concern raised by reviewers. The **Facade** was selected because it precisely matches the problem: there are multiple subsystems (prompt builder, validator, Ollama HTTP client, retry logic) that should be unified behind a single interface callable by the controller.

5.1.3 Architecture Update

Before (baseline):

```javascript
// DashboardController.js -- BEFORE
async generateDietPlan(req, res) {
    const user = await User.findById(req.userId);
    const prompt = this.buildPrompt(user, classification); // 40+ lines
    const response = await axios.post(OLLAMA_URL, { model, prompt });
    const rawText = response.data.response;
    // No validation, no retry
    res.json({ plan: rawText });
}
```

Listing 1: Controller directly calling Ollama – baseline (simplified)

After (delta):

```javascript
// DashboardController.js -- AFTER
const facade = createWellnessPlanFacade(dashboardController);

async generateDietPlan(req, res) {
```

```
    const result = await facade.generateDietPlanResult(
        profile, classification, duration
    );
    if (result.error)
        return res.status(result.statusCode).json({ error: result.error });
    res.json(result.data);
}
```

Listing 2: Controller delegating to Facade – delta

```
class WellnessPlanFacade {
    constructor(deps) {
        this.defaultModel = deps.defaultModel;
        this.planModel    = deps.planModel || 'llama3.1:8b';
        this.generateRecommendation = deps.generateRecommendation;
        this.callOllama             = deps.callOllama;
    }

    async generateStructuredPlan(profile, classification, duration,
                                 planType = 'diet') {
        if (!['daily', 'weekly', 'monthly'].includes(duration))
            return { error: 'Invalid duration', statusCode: 400 };

        const prompt = planType === 'diet'
            ? buildStructuredDietPrompt(profile, classification, duration)
            : buildStructuredExercisePrompt(profile, classification, duration);

        const numPredict = duration === 'monthly' ? 10000
                         : duration === 'weekly'  ? 6000 : 3000;

        for (let attempt = 1; attempt <= 2; attempt++) {
            const finalPrompt = attempt === 1 ? prompt
                : prompt + '\n\nCRITICAL: Previous response was invalid.';
            const rawText = await this.callOllama(
                finalPrompt, this.planModel, numPredict);
            const result = parseAndValidatePlan(
                rawText, duration, getPeriods(duration), planType);
            if (result.valid)
                return { data: { [planType + '_plan']: result.data,
                    metadata: { attempt, source: 'ollama' } },
                    statusCode: 200 };
        }
```

```
        return { error: 'AI plan could not be generated. Please retry.',
                 statusCode: 503 };
    }
}
```

Listing 3: WellnessPlanFacade – core logic (backend/src/facades/WellnessPlanFacade.js)

5.1.4 Updated Component Diagram (Delta)

```
Presentation Layer
  +-- Dashboard / OnboardingSurvey (React)
       +-- POST /dashboard/diet-plan?duration=weekly   (JWT)
       +-- POST /dashboard/exercise-plan?duration=weekly (JWT)

Business Logic Layer
  +-- DashboardController
       +-- WellnessPlanFacade              [NEW -- Facade Pattern]
            +-- planPromptBuilder          [NEW -- utility module]
            |    buildStructuredDietPrompt()
            |    buildStructuredExercisePrompt()
            +-- planValidator              [NEW -- utility module]
            |    parseAndValidatePlan()
            +-- callOllama()              [internal HTTP client]

External Services
  +-- Ollama LLM (:11434)
```

5.1.5 Updated Class Diagram (Delta – new classes only)

```
WellnessPlanFacade
  - defaultModel           : string
  - planModel              : string
  - generateRecommendation : Function (injected)
  - callOllama             : Function (injected)
  -------------------------------------------
  + generateSurveyRecommendation(profile, cls) : Promise
  + generateStructuredPlan(profile, cls,
                           duration, planType) : Promise
  + generateDietPlanResult(profile, cls,
                           duration)          : Promise
  + generateExercisePlanResult(profile, cls,
                               duration)      : Promise

planPromptBuilder (module)
  + getPeriods(duration)                      : string[]
  + buildStructuredDietPrompt(p, cls, dur)    : string
  + buildStructuredExercisePrompt(p, cls, dur): string
```

```
planValidator (module)
  + extractJSON(raw)                          : object|null
  + validatePlanResponse(parsed, pType,
                         periods, planType)   : {valid, data|error}
  + parseAndValidatePlan(raw, pType,
                         periods, planType)   : {valid, data|error}
```

5.2 F2 – Broader Dashboard: Structured Plan Table and Excel Export

5.2.1 Feedback Context

> *"Another mild limitation is that the current dashboard scope is still relatively compact. At this stage, it provides user information and BMI-related output rather than a broader progress-tracking ecosystem."*

5.2.2 Design Decision

Three features were added to address this feedback within the time budget:

1. **Structured Plan Table (`PlanTable` component):** A reusable React component rendering AI-generated plans in an Excel-like grid, supporting daily (1 column), weekly (7 columns), and monthly (30 columns) layouts.
2. **Excel Export:** Users can download any generated plan as an `.xlsx` file via the `xlsx` library.
3. **Duration selector:** Users choose `daily / weekly / monthly` before generating a plan.

A full **progress history** tracking system (F6) was *not* included because it exceeded the budget; see Section 4 rejection justification.

```jsx
export default function PlanTable({ planData, planLabel }) {
  const { periods, rows } = planData;
  return (
    <div className="plan-table-container">
      {planLabel && <h3 className="plan-table-title">{planLabel }</h3>}
      <div className="plan-table-scroll">
        <table className="plan-table-grid">
          <thead>
            <tr>
              <th>Meal / Day</th>
              {periods.map(p => <th key={p}>{p}</th>)}
            </tr>
          </thead>
          <tbody>
            {rows.map(row => (
              <tr key={row.title}>
                <td className="plan-table-row-label">{row.title }</td>
                {row.items.map((cell, i) =>
                  <td key={i}>{cell || '--'}</td>)}
```

```
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    );
}
```

Listing 4: PlanTable component – frontend/src/components/PlanTable.jsx

5.2.3 Updated Sequence Diagram – AI Plan Generation (Delta)

```
User --> OnboardingSurvey (React)
  --> selects duration: "weekly"
  --> clicks "Generate Diet Plan"
  --> loading progress bar starts (animated, 5% to 92%)

  --> POST /api/dashboard/diet-plan?duration=weekly   [JWT]
      --> DashboardController.generateDietPlan()
          --> WellnessPlanFacade.generateDietPlanResult(
                  profile, cls, "weekly")
              --> buildStructuredDietPrompt(profile, cls, "weekly")
              --> callOllama(prompt, "llama3.2:3b", numPredict=6000)
              <-- rawText  (LLM response)
              --> parseAndValidatePlan(rawText, "weekly", periods, "diet")
              <-- { valid: true, data: { periodType, periods, rows } }
          <-- 200 OK { diet_plan: {...}, metadata: { attempt, source } }
      <-- res.json(result.data)

  --> setLoadingProgress(100)  hide loading bar
  --> setDietPlan(data.diet_plan)
  --> <PlanTable planData={dietPlan} planLabel="Weekly Diet Plan" />

User --> clicks "Export to Excel"
  --> XLSX.writeFile(workbook, "diet_plan.xlsx")
```

5.3 F3 – Profile Data to AI Output Continuity

5.3.1 Feedback Context

"Improving the continuity between stored profile data and AI-generated outputs."

At review time, the AI prompt used only a subset of the user's stored profile. Fields such as `allergy_or_restriction`, `diet_preference`, and `screen_hours_per_day` were either absent or only partially included, causing the LLM to generate generic plans that did not respect user-specific constraints.

5.3.2 Design Decision

A dedicated `planPromptBuilder.js` utility was created. Its `buildUserContext()` function maps **all 11 profile fields** into a structured context block injected into every prompt.

```
function buildUserContext(profile, classification) {
    return {
        age:                    profile.age,
        gender:                 profile.gender,
        height_cm:              profile.height_cm,
        weight_kg:              profile.weight_kg,
        bmi:                    classification?.bmi   || 'unknown',
        goal:                   profile.goal,
        diet_preference:        profile.diet_preference || '',
        allergy_or_restriction: profile.allergy_or_restriction || '',
        activity_level:         profile.activity_level,
        exercise_days_per_week: profile.exercise_days_per_week,
        sleep_hours:            profile.sleep_hours,
        level:                  classification?.level || 'unknown'
    };
}
```

Listing 5: buildUserContext – backend/src/utils/planPromptBuilder.js

Allergy/restriction enforcement was added as an **explicit mandatory rule** in the prompt:

```
const userCtx = buildUserContext(profile, classification);
const prompt = `
MANDATORY RULES:
- User restriction: "${userCtx.allergy_or_restriction}" -- NEVER suggest these.
- Diet preference: "${userCtx.diet_preference}" -- strictly follow this.

USER PROFILE:
Age: ${userCtx.age} | Gender: ${userCtx.gender}
Height: ${userCtx.height_cm}cm | Weight: ${userCtx.weight_kg}kg
BMI: ${userCtx.bmi} | Goal: ${userCtx.goal}
Activity: ${userCtx.activity_level}/5
Exercise days/week: ${userCtx.exercise_days_per_week}
Sleep: ${userCtx.sleep_hours}h | Level: ${userCtx.level}
`;
```

Listing 6: Allergy enforcement in prompt – planPromptBuilder.js

Furthermore, the **survey form was updated to pre-populate** from the stored user profile:

```
useEffect(() => {
    const prefillFromProfile = async () => {
```

```
        const res = await axios.get(`${API_URL}/profile`, {
            headers: { Authorization: `Bearer ${token}` }
        });
        const user = res.data?.user;
        if (!user) return;
        setForm(prev => ({
            ...prev,
            age:        prev.age        || String(user.age    ?? ''),
            gender:     prev.gender     || (user.gender || ''),
            height_cm: prev.height_cm || String(user.height ?? ''),
            weight_kg: prev.weight_kg || String(user.weight ?? '')
        }));
    };
    prefillFromProfile();
}, [navigate]);
```

Listing 7: Survey form pre-population – frontend/src/pages/OnboardingSurvey.jsx

5.4 F3b – Strategy Pattern: Level-Specific AI Customization

5.4.1 Feedback Context

This implementation extends F3 feedback ("Profile data to AI output continuity") by adding a critical dimension: beyond mapping all 11 profile fields into the prompt, we now **customize the entire AI recommendation approach** based on the user's fitness level. A beginner receives fundamentally different guidance than an advanced user, and this differentiation is encapsulated using the **Strategy Pattern**.

5.4.2 Design Decision

The **Strategy Pattern** encapsulates level-specific prompt instructions:

- Each fitness level (beginner, intermediate, advanced) has its own strategy class
- Each strategy provides *specialized prompt instructions* tailored to that level
- The strategy is selected at runtime based on user classification
- New levels can be added in the future without modifying existing code

Design Decision: Why Strategy and not if/else in the controller?

Hardcoding level-specific logic with if/else statements would violate the **Open/-Closed Principle** – adding a new fitness level would require touching the controller. The **Strategy Pattern** keeps each level's logic encapsulated in its own class and allows new strategies to be plugged in via `StrategyFactory` without changing existing code. This approach also makes unit testing easier: each strategy can be tested independently.

5.4.3 Implementation

```
const { getStrategy } = require('../strategies/StrategyFactory');

function buildStructuredDietPrompt(profile, classification, duration) {
    // Select strategy based on fitness level
    const strategy = getStrategy(classification.level);
    const levelInstruction = strategy.getLevelSpecificInstruction();

    // Build user context with all 11 fields
    const userCtx = buildUserContext(profile, classification);

    // Inject strategy-specific instruction into prompt
    const prompt = `
USER PROFILE:
Age: ${userCtx.age} | Gender: ${userCtx.gender}
Height: ${userCtx.height_cm}cm | Weight: ${userCtx.weight_kg}kg
BMI: ${userCtx.bmi} | Goal: ${userCtx.goal}
Activity: ${userCtx.activity_level}/5

LEVEL-SPECIFIC INSTRUCTION:
${levelInstruction}

MANDATORY RULES:
- Restriction: "${userCtx.allergy_or_restriction}" -- NEVER suggest these.
- Diet preference: "${userCtx.diet_preference}" -- strictly follow this.

DIET PLAN for ${duration} duration:
...
`;
    return prompt;
}
```

Listing 8: Strategy integration in prompt builder – backend/src/utils/planPromptBuilder.js

```
const { getStrategy } = require('../strategies/StrategyFactory');

async generateDietPlan(req, res) {
    const profile = await User.findById(req.userId);
    const classification = this.classifyUser(surveyData);

    // Generate plan via Facade
    const result = await facade.generateDietPlanResult(
        profile, classification, duration
    );

    // If Ollama fails, use strategy's fallback recommendation
```

```
    if (result.error) {
        const strategy = getStrategy(classification.level);
        result.data = { diet_plan: strategy.
    getDefaultRecommendation() };
        result.statusCode = 200;
    }

    res.json(result.data);
}
```

Listing 9: Strategy selection in controller – backend/src/controllers/dashboardController.js

5.4.4 Strategy Classes

Each concrete strategy provides level-appropriate recommendations:

```
BeginnerStrategy
  → "Temel, düşük yoğunluklu ve kolay uygulanabilir tut..."
  → Fallback: 6-section beginner plan with basic guidelines

IntermediateStrategy
  → "Güçlendirme, dayanıklılık ve sürdürülebilirlik odaklı..."
  → Fallback: 6-section intermediate plan with progressive structure

AdvancedStrategy
  → "Performans ve ilerleme odaklı yapı..."
  → Fallback: 6-section advanced plan with specialized guidance
```

5.4.5 Quality Impact

Strategy Pattern integration ensures:

- **Correctness**: Plans match the user's fitness level expectations
- **Maintainability**: New levels can be added without modifying Facade or controller
- **Resilience**: If Ollama fails, fallback recommendations are level-appropriate, not generic
- **Testability**: Each strategy can be unit-tested independently

5.5 F4 – End-to-End AI Validation (`planValidator.js`)

5.5.1 Feedback Context

> *"There is also room for future enhancement in areas such as richer end-to-end validation around AI-assisted flows."*

At review time, if Ollama returned malformed JSON, incomplete arrays, or injected personal data back into plan cells, the frontend would either crash or display garbage to the user.

5.5.2 Design Decision

A dedicated `planValidator.js` module was created implementing a **3-stage validation pipeline**: Extract → Validate & Repair → Sanitize. Additionally, an Ollama fallback mechanism was added at the Facade level.

```
function extractJSON(raw) {
    // 1. Direct parse
    try { return JSON.parse(raw.trim()); } catch (_) {}

    // 2. Extract from markdown code fence
    const fenceMatch = raw.trim().match(/```(?:json)?\s*([\s\S]*?)```/);
    if (fenceMatch) {
        try { return JSON.parse(fenceMatch[1].trim()); } catch (_) {}
    }

    // 3. Find first '{' ... last '}'
    const first = raw.indexOf('{'), last = raw.lastIndexOf('}');
    if (first !== -1 && last > first) {
        try { return JSON.parse(raw.slice(first, last + 1)); } catch (_) {}
    }
    return null;
}
```

Listing 10: JSON extraction from raw LLM text – planValidator.js

```
function validatePlanResponse(parsed, periodType, expectedPeriods, planType) {
    const expectedTitles = planType === 'diet'
        ? DIET_ROW_TITLES : EXERCISE_ROW_TITLES;

    // Repair row count (pad or trim)
    while (parsed.rows.length < expectedTitles.length)
        parsed.rows.push(
            { title: expectedTitles[parsed.rows.length], items: [] });
    parsed.rows = parsed.rows.slice(0, expectedTitles.length);

    // Repair items per row
    for (const row of parsed.rows) {
        if (row.items.length < expectedPeriods.length) {
            const existing = row.items.filter(c => c && c !== '-');
            while (row.items.length < expectedPeriods.length)
                row.items.push(
                    existing[row.items.length % existing.length] || '');
        }
        row.items = row.items.slice(0, expectedPeriods.length);
```

```
    }

    // Sanitize: remove cells with leaked personal info
    const personalPatterns = [
        /\b(yas|cinsiyet|boy|kilo|bmi|hedef)\s*:/i,
        /\b\d{2,3}\s*(kg|cm)\b/i
    ];
    for (const row of parsed.rows)
        for (let j = 0; j < row.items.length; j++)
            if (personalPatterns.some(p => p.test(row.items[j]))
                row.items[j] = '';

    // Reject if >50% cells empty
    const total = parsed.rows.reduce((s, r) => s + r.items.length, 0);
    const empty = parsed.rows.reduce(
        (s, r) => s + r.items.filter(c => !c || c === '-').length, 0);
    if (total > 0 && empty / total > 0.5)
        return { valid: false,
                 error: Math.round(empty / total * 100) + '% cells empty.' };

    return { valid: true, data: { periodType, periods: expectedPeriods,
        rows: parsed.rows.map((r, i) =>
            ({ title: expectedTitles[i], items: r.items })) }};
}
```

Listing 11: Validation and repair logic – planValidator.js (excerpt)

The **retry mechanism** inside `WellnessPlanFacade` executes up to 2 attempts. If both fail, the Facade returns `HTTP 503` with a user-friendly message – the application remains functional.

5.6 F2b – Observer Pattern: Real-Time Notification System

5.6.1 Feedback Context

This implementation extends the F2 feedback ("Broader dashboard capabilities") with a second dimension: rather than only displaying static plan output, the dashboard now **notifies the user in real time** whenever a significant wellness event occurs. This was motivated by the reviewers' observation that the dashboard lacked interactivity and progress-awareness beyond BMI.

5.6.2 Design Decision

The **Observer Pattern** was chosen to decouple event producers (plan generation, profile updates, survey completion) from event consumers (the notification persistence layer and the frontend UI). This allows new notification types or new observer implementations to be added in the future without modifying any controller or plan-generation logic.

Design Decision: Why Observer and not direct DB inserts in the controller?

Inserting notifications directly inside `DashboardController` would violate the **Single Responsibility Principle** and create tight coupling: the controller would need to know about the notification schema. The **Observer Pattern** keeps the controller unaware of who reacts to its events; it only calls `emitPlanCreated()`, and any number of observers can respond. This also makes the system **extensible**: future observers (e.g., email, push notification) can be attached without touching existing code.

5.6.3 Class Structure

The implementation follows the classic GoF Observer pattern with abstract base classes:

```
<<abstract>> Observer
  + update(eventData) : void      [throws if not overridden]

<<abstract>> Subject
  - observers : Observer[]
  + attach(observer)  : void
  + detach(observer)  : void
  + notify(eventData) : Promise<void>
  + getState()        : object    [throws if not overridden]

PlanEventEmitter  extends  Subject
  - currentEvent : object
  + emitPlanCreated(userId, planType, duration, routineId) : Promise
  + emitPlanUpdated(userId, planType)                      : Promise
  + emitProfileUpdated(userId)                             : Promise
  + emitSurveyCompleted(userId)                            : Promise
  + emitGoalMissed(userId, goalType)                       : Promise
  + getState()                                             : object

UserNotificationObserver  extends  Observer
  - userId : string
  + update(eventData) : Promise   [INSERTs into notifications table]
```

5.6.4 Database Schema Extension

A new `notifications` table was added to PostgreSQL to persist observer-generated events:

```sql
CREATE TABLE IF NOT EXISTS notifications (
    notification_id UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id         UUID          NOT NULL
                    REFERENCES users(user_id) ON DELETE CASCADE,
    type            VARCHAR(50)   NOT NULL,
```

```sql
    title           VARCHAR(255) NOT NULL,
    message         TEXT         NOT NULL,
    reference_id    UUID,
    is_read         BOOLEAN      NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    read_at         TIMESTAMPTZ
);

CREATE INDEX idx_notifications_unread
    ON notifications (user_id, is_read) WHERE is_read = FALSE;
```

Listing 12: notifications table – backend/schema.sql

5.6.5 Implementation

```javascript
class UserNotificationObserver extends Observer {
    constructor(userId) {
        super();
        this.userId = userId;
    }

    async update(eventData) {
        const query = `
            INSERT INTO notifications
                (notification_id, user_id, type, title,
                 message, reference_id)
            VALUES ($1, $2, $3, $4, $5, $6)
            RETURNING *;
        `;
        const result = await pool.query(query, [
            uuidv4(),
            this.userId,
            eventData.type,
            eventData.title,
            eventData.message,
            eventData.referenceId || null
        ]);
        return result.rows[0];
    }
}
```

Listing 13: UserNotificationObserver – concrete Observer (backend/src/observers/UserNotificationObserver.js)

```javascript
// Per-request: create observer for this user, attach to
    emitter
const observer = new UserNotificationObserver(req.userId);
planEventEmitter.attach(observer);

// After plan generation succeeds:
await planEventEmitter.emitPlanCreated(req.userId, planType,
    duration);
```

```

// Clean up: detach so observer does not receive future events
planEventEmitter.detach(observer);
```

Listing 14: Observer wired into DashboardController (simplified)

5.6.6 Frontend Notification System

The frontend consumes the notification API via a React Context and custom hook architecture:

- **`NotificationContext.jsx`**: Global state provider – holds notifications array and unread count; exposes `addNotification`, `markAsRead`, `removeNotification`
- **`NotificationCenter.jsx`**: Bell icon with unread badge; opens the inbox panel
- **`NotificationInbox.jsx`**: Scrollable list of all notifications; mark-as-read on click
- **`NotificationToast.jsx`**: Auto-dismissing toast (3 seconds) after each new notification; colour-coded by event type
- **`notificationAPI.js`**: HTTP wrappers for the notifications REST endpoints (get all, get unread count, mark as read)

5.6.7 Supported Event Types

Table 5: Observer event types and their triggers

Event Type	Trigger	Toast Colour
`PlanCreated`	Diet or exercise plan successfully generated	Green
`PlanUpdated`	Existing plan regenerated or modified	Blue
`ProfileUpdated`	User saves changes in Edit Profile page	Orange
`SurveyCompleted`	Onboarding health survey submitted	Green
`GoalMissed`	User-defined goal threshold not met	Red

5.6.8 Observer Pattern Sequence Diagram

```
User --> clicks "Generate Diet Plan"
  --> POST /api/dashboard/diet-plan?duration=weekly  [JWT]
      --> DashboardController
          --> observer = new UserNotificationObserver(req.userId)
          --> planEventEmitter.attach(observer)
          --> WellnessPlanFacade.generateDietPlanResult(...)
          <-- { data: { diet_plan: {...} }, statusCode: 200 }
          --> planEventEmitter.emitPlanCreated(userId, "diet", "weekly")
              --> Subject.notify(eventData)
                  --> UserNotificationObserver.update(eventData)
                      --> INSERT INTO notifications (...) RETURNING *
                  <-- notification record
          --> planEventEmitter.detach(observer)
```

```
      <-- res.json({ diet_plan: {...} })

Frontend (NotificationContext)
  --> addNotification(notification)
  --> setUnreadCount(prev + 1)
  --> <NotificationToast /> appears for 3 seconds
  --> Bell icon badge updates
```

5.7 F7 – Documentation Alignment

5.7.1 Feedback Context

> *"Continued refinement of documentation so that setup and implementation details remain aligned."*

5.7.2 What Was Updated

The SDD documents (v1–v3) were produced incrementally during earlier sprint phases and are kept as versioned snapshots. Rather than modifying those baseline files, all documentation alignment for the delta phase is consolidated in this report. The following updates are therefore recorded here:

- **Component diagram (delta)** – updated in Section **??** (F1 architecture update): `WellnessPlanFacade`, `planPromptBuilder`, and `planValidator` added as named nodes under the Business Logic layer.
- **Class diagram (delta)** – updated in Section **??**: full `WellnessPlanFacade` interface and the complete Observer hierarchy (`Observer`, `Subject`, `PlanEventEmitter`, `UserNotificationObserver`) documented.
- **Sequence diagram (delta)** – updated in Sections **??** and **??**: AI plan generation flow now reflects the Facade delegation, validation pipeline, and Observer notification chain.
- **Design patterns table** – updated in Section 6: Facade and Observer entries added alongside the four baseline patterns from SDD v3.
- **Ollama model name** – SDD v3 recorded `llama3.1:8b` as the planned model. The actual runtime model used is `llama3.2:3b`; this discrepancy is noted here and reflected in the Facade implementation (Bug B2 fix, Section 9).
- **Notifications API** – new `/api/notifications` endpoints and the `notifications` database table are documented in Section **??** and the Appendix file list; they postdate SDD v3.

6 Architecture Summary: Before vs. After

Table 6: Architecture comparison: baseline vs. delta

Concern	Before (at review)	After (delta)
AI architecture	Python client (separate process, not web-integrated)	`WellnessPlanFacade` in Node.js backend
AI concern separation	120+ lines mixed into `DashboardController`	Controller: ∼30 lines; Facade handles all AI
Output validation	None – raw LLM text shown to user	3-stage pipeline: extract, repair, sanitize
Ollama failure	Unhandled exception / crash	HTTP 503 + user message (graceful degradation)
Plan format	Unstructured plain text (markdown)	Structured JSON rendered as grid table in UI
Plan export	Not available	Excel (.xlsx) download
Plan duration	Fixed (single output)	User-selectable: daily / weekly / monthly
Survey form	Empty placeholders for returning users	Pre-populated from stored profile
JWT logout	Token removed from localStorage only	Token blacklisted server-side
Notifications	None	Real-time toast + inbox via Observer Pattern
DB schema	`users` table only	+ `notifications` + `routines` tables
Design patterns	Builder ×2, MVC, Middleware	+ Facade + Observer

7 New Design Patterns Added

7.1 Strategy Pattern (Enhanced at Delta Phase)

7.1.1 Overview

The **Strategy Pattern** was already implemented at baseline to encapsulate fitness-level-specific AI prompt customization. During the delta phase, it was deeply integrated with the Facade (F1) and Profile Continuity (F3/F3b) features to ensure level-appropriate recommendations across all user segments.

7.1.2 Class Structure

```
<<abstract>> BaseStrategy
  - levelKey : 'beginner'|'intermediate'|'advanced'
  + getLevelKey()                  : string
  + getLevelLabel()                : string (Turkish)
  + getLevelSpecificInstruction()  : string      [abstract]
  + applyToPrompt(prompt, profile, classification) : string
```

```
  + getDefaultRecommendation()    : string      [abstract]

BeginnerStrategy   extends  BaseStrategy
  + getLevelSpecificInstruction() : "Temel, düşük yoğunluklu..."
  + getDefaultRecommendation()   : 6-section beginner plan

IntermediateStrategy extends BaseStrategy
  + getLevelSpecificInstruction() : "Güçlendirme, dayanıklılık..."
  + getDefaultRecommendation()   : 6-section intermediate plan

AdvancedStrategy   extends BaseStrategy
  + getLevelSpecificInstruction() : "Performans ve ilerleme..."
  + getDefaultRecommendation()   : 6-section advanced plan

StrategyFactory (module)
  + getStrategy(level) : BaseStrategy instance
  + normalizeFitnessLevel(level) : string
```

7.1.3 Purpose

At runtime, when generating an AI plan via the Facade, the system selects the appropriate strategy based on the user's fitness level classification (`beginner`, `intermediate`, or `advanced`):

- **BeginnerStrategy**: Generates low-intensity, foundational prompt instructions for users new to wellness routines.
- **IntermediateStrategy**: Provides moderate-challenge prompt instructions balancing strength, cardio, and progressive overload.
- **AdvancedStrategy**: Delivers high-intensity, specialized prompt instructions for users with established fitness habits.

Each strategy encapsulates its own `getLevelSpecificInstruction()` block, which is appended to the Facade's AI prompt during plan generation. This ensures the LLM receives level-appropriate context without mixing concerns in the controller or Facade.

7.1.4 Integration with Facade and Profile Continuity

The Strategy Pattern integrates seamlessly with the Facade Pattern (F1) and Profile Continuity (F3/F3b):

```
// Inside WellnessPlanFacade.generateStructuredPlan()
const { getStrategy } = require('../strategies/StrategyFactory');
const strategy = getStrategy(classification.level);
const levelInstruction = strategy.getLevelSpecificInstruction();

const prompt = buildStructuredDietPrompt(
    profile,
    classification,
```

```
    duration
);
// levelInstruction already injected into prompt by builder
```

Listing 15: Strategy selected in Facade pipeline

7.1.5 Fallback Recommendations

If Ollama becomes unavailable, each strategy provides a level-appropriate fallback recommendation via `getDefaultRecommendation()`. This ensures graceful degradation: users receive recommendations tailored to their fitness level, not generic fallbacks.

Table 7: Strategy pattern – summary

Attribute	Detail
Pattern	Strategy
Classes	`BaseStrategy`, `BeginnerStrategy`, `IntermediateStrategy`, `AdvancedStrategy`
Factory	`StrategyFactory.getStrategy(level)`
Files	`backend/src/strategies/` (5 files)
Problem solved	Different fitness levels require fundamentally different AI recommendations; hardcoding would violate OCP
Solution	Encapsulate level-specific prompt instructions in concrete strategy classes; select at runtime via factory
Related feedback	F3 (profile continuity), F3b (level-specific customization)
Related req.	REQ-AI-02, REQ-AI-06
Quality impact	Correctness ↑ (level-appropriate plans), Maintainability ↑ (new levels w/o controller changes), Resilience ↑ (level-specific fallback)

7.2 Facade Pattern

Table 8: Facade pattern – summary

Attribute	Detail
Pattern	Facade
Class	`WellnessPlanFacade`
File	`backend/src/facades/WellnessPlanFacade.js`
Problem solved	`DashboardController` overloaded with AI orchestration, prompt building, Ollama HTTP calls, and retry logic – violating SRP
Solution	Single interface hides all internal complexity; controller becomes a thin delegator
Related feedback	F1, F4
Related req.	REQ-AI-01, REQ-AI-03, REQ-ARCH-02
Quality impact	Maintainability ↑, Testability ↑, Reliability ↑

7.3 Observer Pattern

Table 9: Observer pattern – summary

Attribute	Detail
Pattern	Observer (GoF)
Subject class	`PlanEventEmitter` (extends `Subject`)
Observer class	`UserNotificationObserver` (extends `Observer`)
Files	`backend/src/observers/` (4 files)
Problem solved	Controllers tightly coupled to notification logic; adding new notification channels would require modifying controllers
Solution	Controllers emit events; observers react independently; new observers can be attached without touching existing code
Related feedback	F2 (broader dashboard)
Related req.	REQ-UI-06, REQ-ARCH-04, REQ-ARCH-05
Quality impact	Extensibility ↑, Coupling ↓, Usability ↑

7.4 Complete Design Pattern Summary (Post-Delta)

Table 10: All design patterns in the system after delta implementation

Pattern	Class(es)	Purpose	Status
Builder	`ProfileBuilder`	Profile update payload validation and step-by-step assembly	Existing
Builder	`SqlUpdateBuilder`	Dynamic SQL UPDATE SET clause via method chaining	Existing
MVC	Controller + Model + Route	Separation of routing, business logic, data access	Existing
Middleware	`JwtMiddleware`	Cross-cutting JWT authentication for protected routes	Existing
Strategy	`BaseStrategy`, `BeginnerStrategy`, `etc.`	Fitness-level-specific prompt customization; selectable via factory	**Enhanced**
Facade	`WellnessPlanFacade`	Unified interface for AI orchestration, hiding subsystem complexity	**New**
Observer	`PlanEventEmitter`, `UserNotification-Observer`	Decoupled event-driven notifications for plan/profile/survey events	**New**

8 Quality Factor Impact

Table 11: Quality factors: metric, target, and delta impact

Factor	**Criterion / Metric**	**Req.**	**Before**	**After**	**Target**	**Test**
Reliability	System available when Ollama is down	REQ-AI-03	Crash	HTTP 503 + msg	Graceful	TC-AI-05[a]
Correctness	AI plan respects diet restrictions	REQ-AI-01	Inconsistent	Enforced	Always	TC-AI-02[b]
Correctness	All 11 profile fields used in prompt	REQ-AI-02	4 fields	11 fields	≥ 8	TC-AI-01[c]
Maintainability	AI lines in DashboardController	REQ-ARCH-02	∼120	∼30 lines	< 50	Code rev.
Maintainability	SRP satisfaction (Facade)	REQ-ARCH-02	Violated	Satisfied	Satisfied	Code rev.
Testability	AI module independently testable (DI)	REQ-ARCH-03	No	Yes	Yes	TC-AI-07[d]
Usability	Plan in structured grid format	REQ-UI-04	Plain text	Grid table	Grid	TC-UI-03[e]
Functionality	Export plan to external format	REQ-UI-05	None	.xlsx export	Export	TC-UI-04[f]
Robustness	Invalid LLM JSON handled	REQ-AI-04	Crash	Repair+retry	Repair	TC-AI-03[g]
Security	JWT invalidated server-side on logout	REQ-SEC-01	Client-only	Blacklisted	Server	TC-SEC-01[h]
UX	Survey pre-filled for returning users	REQ-UI-03	Empty	Pre-populated	Pre-filled	TC-UI-02[i]
Extensibility	New notification types w/o ctrl change	REQ-ARCH-04	Not possible	Observer attach	Pluggable	Code rev.
Usability	Real-time feedback on wellness events	REQ-UI-06	None	Toast + inbox	Present	TC-UI-06[e]
Coupling	Event producers decoupled from consumers	REQ-ARCH-05	Tight	Loose (Obs.)	Loose	Code rev.

[a] `validatePlanResponse rejects payloads with too many empty cells` (fallback path, unit test suite 41/41 pass) [b] `diet prompt omits restriction rule when allergy info is absent` [c] `diet`

`prompt includes periods titles and restriction guidance`; `exercise prompt includes exercise rows and serialized user context` [d] `parseAndValidatePlan runs extraction and normalization together` [e] Selenium smoke test: dashboard load and plan generation flow confirmed [f] Frontend production build validation: `xlsx` dependency bundled successfully [g] `extractJSON parses raw JSON markdown fences and surrounding text`; `validatePlanResponse sanitizes leaked personal information` [h] `auth middleware rejects blacklisted tokens` [i] Selenium smoke test: profile fields visible and pre-populated after login

9 Effort Actuals vs. Estimates

Table 12: Effort actuals compared to PERT estimates

#	Feedback	Estimated	Actual	Δ	Notes
F1	Facade Pattern	4.2 hrs	4.5 hrs	+0.3	Dependency injection wiring took longer than expected
F2	Plan Table + Excel	3.0 hrs	3.5 hrs	+0.5	Monthly 30-column layout required extra CSS scrolling work
F3	Prompt continuity	2.0 hrs	1.5 hrs	-0.5	Simpler once prompt builder was structured
F4	AI validation	2.2 hrs	2.5 hrs	+0.3	Regex sanitisation patterns needed multiple iterations
F7	Documentation	1.2 hrs	1.0 hrs	-0.2	SDD update was straightforward
Total		**12.6 hrs**	**13.0 hrs**	**+0.4**	∼3% overrun – within acceptable range

Observation: The PERT estimation was accurate overall. The slight overrun on F1 and F2 was offset by underruns on F3 and F7. The total effort of 13.0 hours distributed across 5 days at an average of 2.6 hours/day remained within the planned budget.

10 Bug Fix Log (Testing Phase)

The following bugs were discovered and fixed during the dedicated testing and bug-fix phase (last 1–2 days of the implementation window):

Table 13: Bug fix log – testing phase

#	Bug	Severity	Root Cause	Fix Applied	Verified By
B1	Exercise plan button non-functional	Medium	Missing route handler for `/exercise-plan`	Handler added and wired to `WellnessPlanFacade`	TC-AI-06 – *exercise prompt* unit test + Selenium smoke
B2	Ollama model name mismatch (`llama3.1:8b` hardcoded)	High	`DEFAULT_MODEL` env variable not read; wrong model called at runtime	`DEFAULT_MODEL` reads from `process.env` with `llama3.2:3b` fallback	TC-AI-07 – *parseAndValidatePlan* unit test
B3	Survey form empty for returning users	Low	Form did not pre-populate fields from stored profile	`useEffect` added to pre-fill `height/weight/age/gender`; `data-testid` selectors added to `OnboardingSurvey.jsx` and `Dashboard.jsx` for Selenium coverage	TC-UI-02 – Selenium (profile fields visible after login)
B4	JWT token not invalidated server-side on logout	High	Token only removed from `localStorage`; blacklisted tokens accepted by middleware	In-memory `tokenBlacklist` Set introduced; `auth` middleware checks blacklist before accepting token	TC-SEC-01 – *auth middleware rejects blacklisted tokens*
B5	AI plan cells showing personal profile info (age, weight, BMI)	Medium	LLM ignored prompt-level instruction not to leak personal data	Regex sanitization patterns added in `planValidator.validatePlanResponse`; matched cells replaced with empty string	TC-AI-03 – *validatePlanResponse sanitizes leaked personal information*

(continued)

#	Bug	Severity	Root Cause	Fix Applied	Verified By
B6	Loading progress bar stuck at 92% after plan response	Low	Progress interval not cleared when API response arrived	`setLoadingProgress(100)` called in `finally` block so it fires on both success and error paths	TC-UI-05 – Selenium smoke (plan generation flow completed without UI freeze)

All six bugs were resolved before the final demo build. The backend unit test suite remained green at **41/41** after each fix, confirmed by running `npm test` in the `backend/` directory. The Selenium end-to-end smoke test validated the full *registration* → *dashboard* → *profile update* → *logout* → *re-login* flow successfully after bug fixes B3 and B6 were applied. Full test output is recorded in the project's **Test Report** (2026-04-14).

11 Conclusion

Five of the seven feedbacks received from the peer review were selected for implementation. The selection was driven by **effort-to-value ratio**: the chosen feedbacks (F1, F2, F3, F4, F7) collectively addressed the reviewers' most critical concern – the disconnection between the AI module and the web application – while remaining within the available 13-hour implementation budget.

Two new design patterns were introduced as a direct result of addressing the review feedback:

- The **Facade Pattern** (`WellnessPlanFacade`) resolved the primary architectural weakness: AI logic is now fully integrated into the Node.js web flow with structured validation and retry logic.
- The **Observer Pattern** (`PlanEventEmitter` + `UserNotificationObserver`) extended the dashboard with a real-time notification system, decoupling event production from event consumption and enabling future extensibility.

Two feedbacks were deferred:

- **F5 (Deployment readiness):** Estimated at 8+ hours; deferred to a post-course production phase.
- **F6 (Progress tracking):** Estimated at 10+ hours; requires new database schema, API endpoints, and frontend pages – deferred to a future sprint.

The implementation window was used as planned: feedbacks were addressed in the first 3 days, followed by 2 days of integration testing and bug fixing, resulting in a stable, demo-ready system with 6 documented bugs identified and resolved.

A File Structure Reference

Table 14: Modified and created files during delta implementation

	File	Change
New	`backend/src/facades/WellnessPlanFacade.js`	Facade Pattern – AI orchestration
New	`backend/src/utils/planPromptBuilder.js`	Structured prompt generation
New	`backend/src/utils/planValidator.js`	LLM output validation and repair
New	`backend/src/observers/Observer.js`	Abstract Observer base class
New	`backend/src/observers/Subject.js`	Abstract Subject: attach / detach / notify
New	`backend/src/observers/PlanEventEmitter.js`	Concrete Subject – 5 event types
New	`backend/src/observers/UserNotificationObserver.js`	Concrete Observer – persists to DB
New	`frontend/src/components/PlanTable.jsx`	Reusable plan grid component
New	`frontend/src/context/NotificationContext.jsx`	Global notification state (React Context)
New	`frontend/src/components/NotificationCenter.jsx`	Bell icon + unread badge
New	`frontend/src/components/NotificationInbox.jsx`	Notification list panel
New	`frontend/src/components/NotificationToast.jsx`	Auto-dismiss toast (3 seconds)
New	`frontend/src/api/notificationAPI.js`	API wrappers for notification endpoints
New	`frontend/src/hooks/useNotifications.js`	Custom React hook
New	`frontend/src/styles/NotificationCenter.css`	Notification centre styles
New	`frontend/src/styles/NotificationInbox.css`	Notification inbox styles
New	`frontend/src/styles/NotificationToast.css`	Toast styles
New	`frontend/src/styles/PlanTable.css`	Plan table styles
Mod	`backend/src/controllers/dashboardController.js`	Delegated to Facade; Observer wired in
Mod	`backend/src/middleware/auth.js`	Token blacklist check added
Mod	`backend/src/utils/tokenBlacklist.js`	In-memory blacklist Set
Mod	`backend/schema.sql`	Added notifications and routines tables
Mod	`frontend/src/pages/OnboardingSurvey.jsx`	Profile pre-fill, plan table, Excel export
Mod	`docs/architecture/SDD_v3.md`	Diagrams and patterns updated
Mod	`README.md`	Ollama model name corrected